

**Пермский институт (филиал) федерального
государственного бюджетного образовательного
учреждения высшего образования
«Российский экономический университет имени Г.В.
Плеханова»**

Кафедра информационных технологий и программирования.

Сопроводительная записка к практической работе №1.

Тема работы: «Создание и запуск модульных тестов для
управляемого кода»

Работу выполнила:
Александрович Валерия
Николаевна
Группа: ИПс-11
Преподаватель: Берестов
Дмитрий Борисович

Пермь 2026

Оглавление.

Введение.....	3
1. Создание проекта для тестирования.	4
2. Создание проекта модульного теста.	10
3. Создание тестового класса.....	14
4. Создание метода теста.....	16
5. Сборка и запуск теста.....	17
6. Исправление кода и повторный запуск тестов.	20
7. Создание и запуск новых методов теста.	21
8. Рефакторинг тестируемого кода.....	22
9. Рефакторинг тестовых методов.....	23
10. Повторное тестирование, переписывание и анализ.	24
Заключение.	26

Введение.

Выполняя практическую работу №1 по теме «Создание и запуск модульных тестов для управляемого кода», я ознакомилась и изучила учебное пособие Средства тестирования Visual Studio.

Практическая работа №1 была выполнена по руководству "Средства тестирования Visual Studio-2022", по стр. 158 -170.

(<https://cloud.mail.ru/public/JaXA/BUKbRzZoN>).

1. Создание проекта для тестирования.

1. Я запустила Visual Studio.

Visual Studio — это интегрированная среда разработки (IDE), разработанная Microsoft. Она используется для разработки компьютерных программ, в том числе веб-сайтов, веб-приложений, веб-сервисов и мобильных приложений. Visual Studio использует платформы разработки программного обеспечения Microsoft, в том числе Windows API, Windows Forms, Windows Presentation Foundation (WPF), Microsoft Store и Microsoft Silverlight. Он может генерировать как собственный код, так и управляемый код.

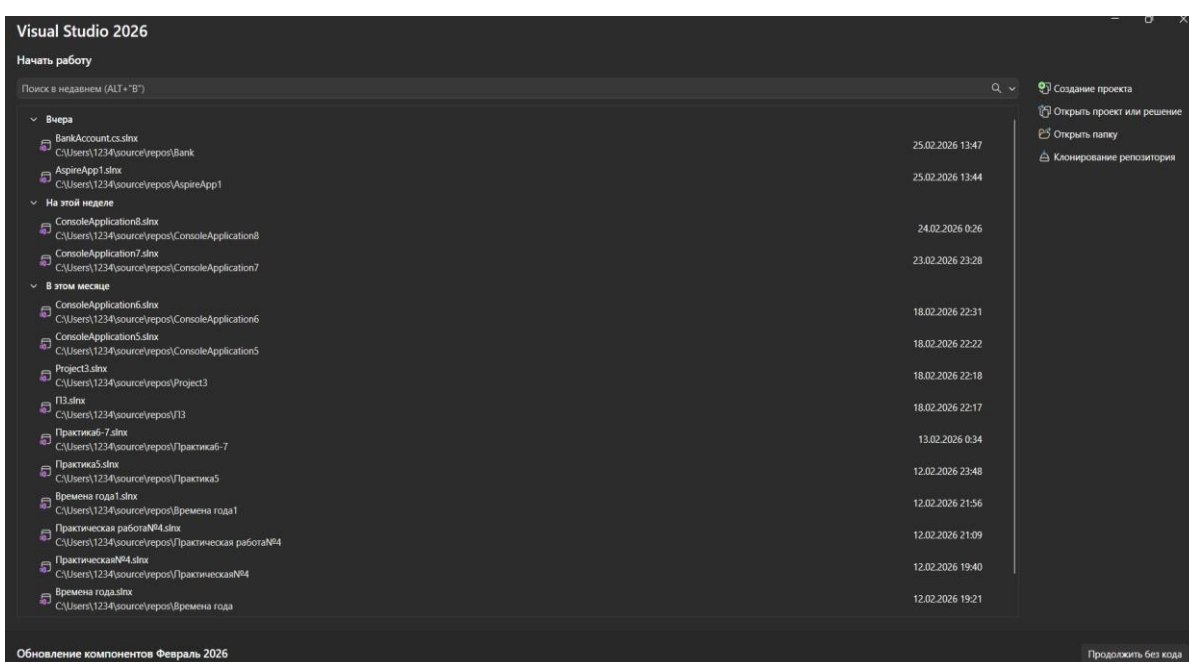


Рисунок 1

2. Я создала проект.

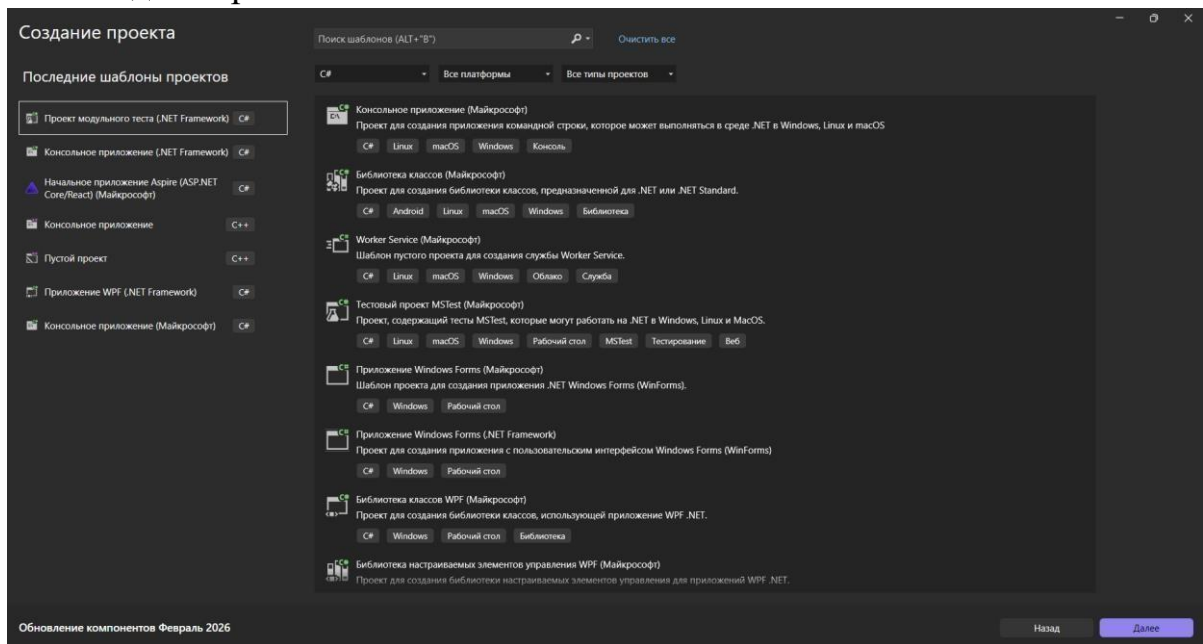


Рисунок 2

3. Я выбрала шаблон проекта Консольное приложение на C#.

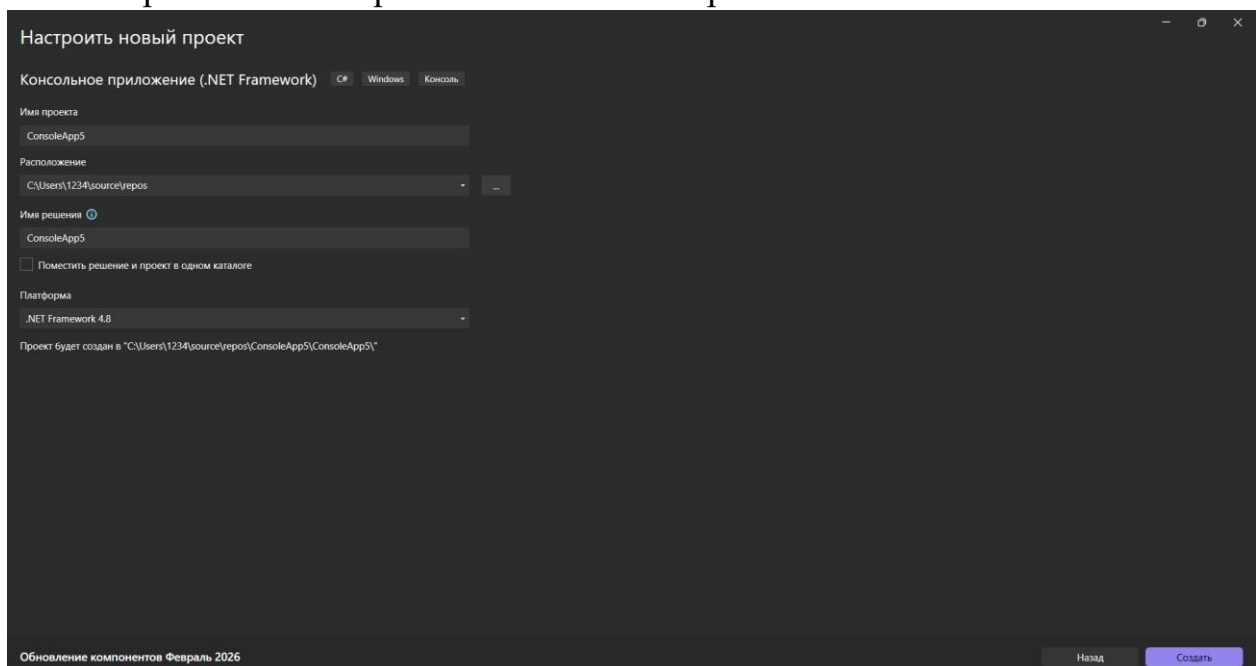


Рисунок 3

4. Я назвала проект Bank и выбрала рекомендуемую версию целевой платформы.

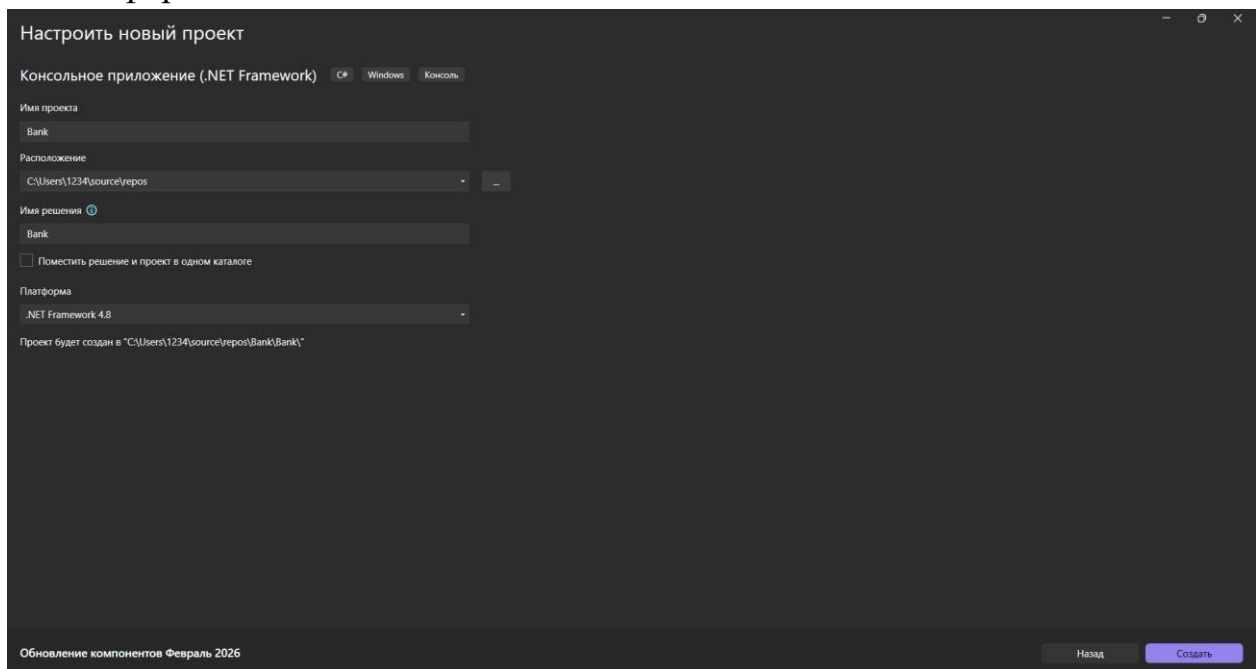


Рисунок 4

5. Я заменила содержимое файла Program.cs предложенным кодом на C#, который определяет класс BankAccount.

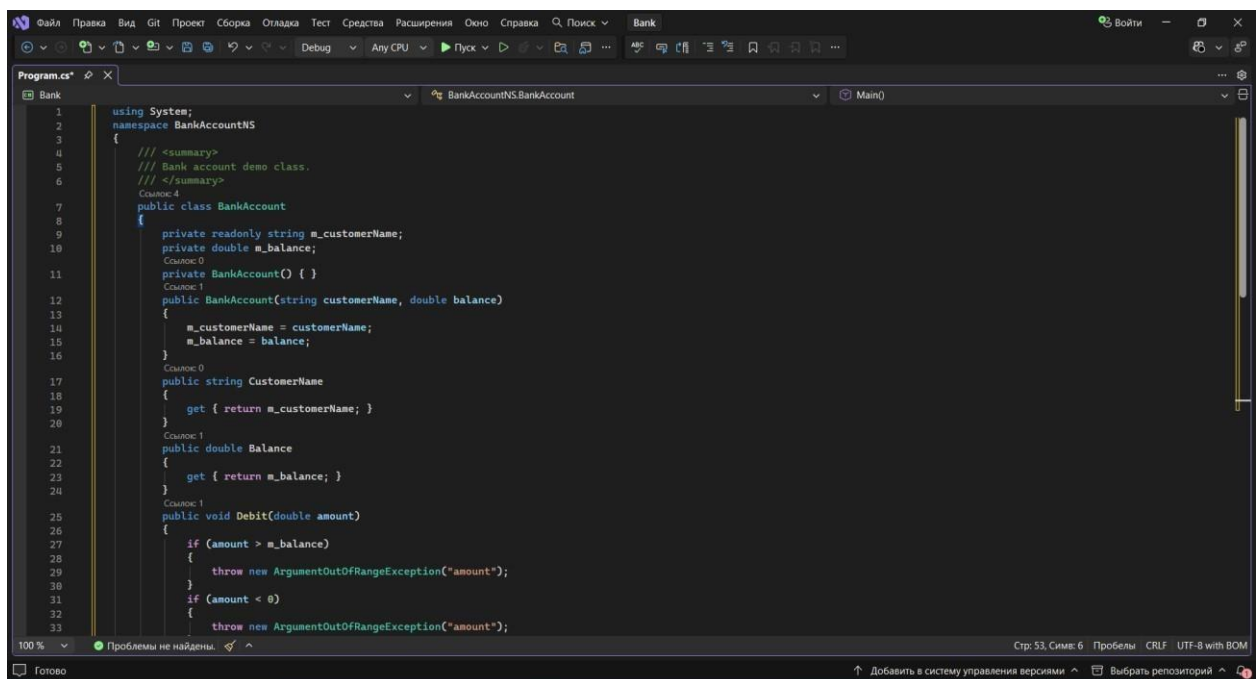


Рисунок 5

6. Я переименовала файл в BankAccount.cs.

Для этого я зашла в Обзорщик решений, нажала на Bank правой кнопки мыши и выбрала Переименовать.

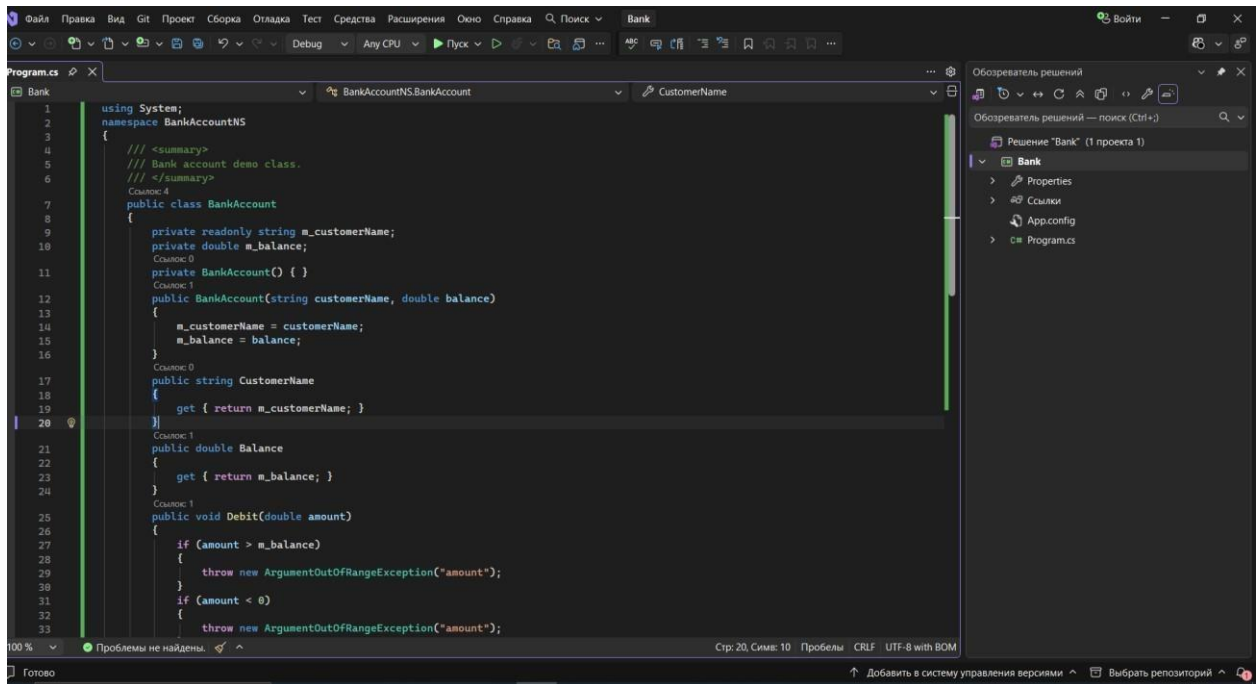


Рисунок 6

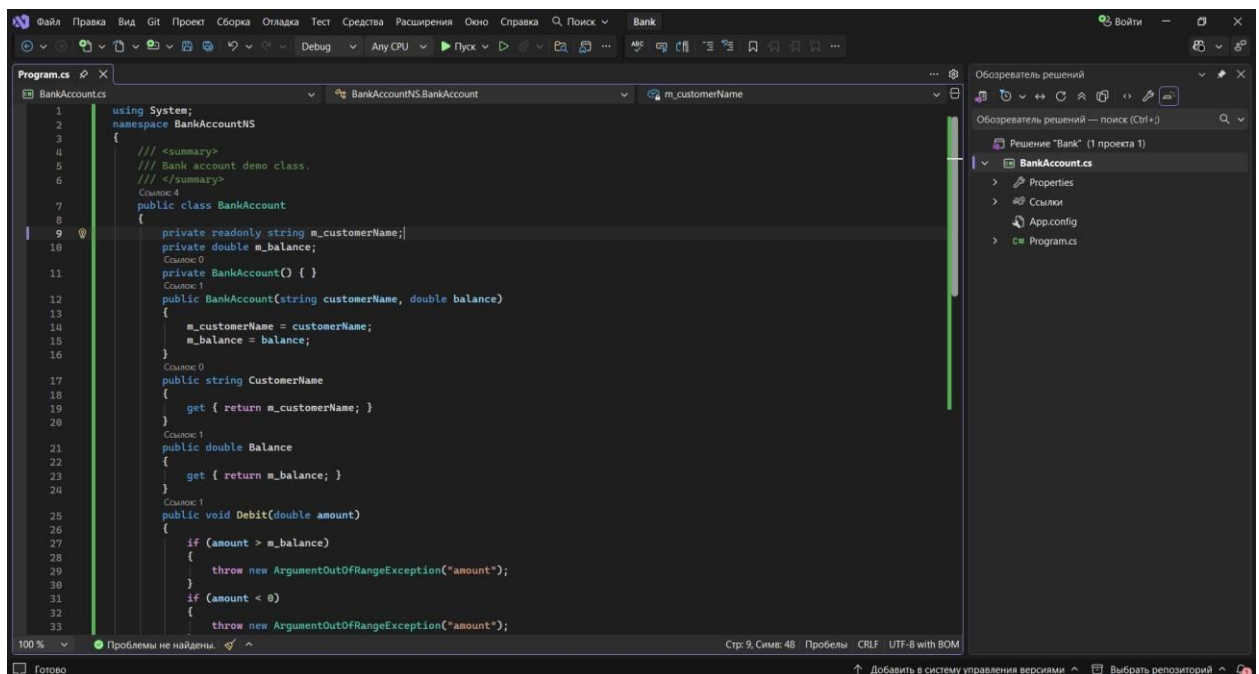


Рисунок 7

7. Я использовала клавиши CTRL +SHIFT + B для того, чтобы произошла сборка (вывод был).

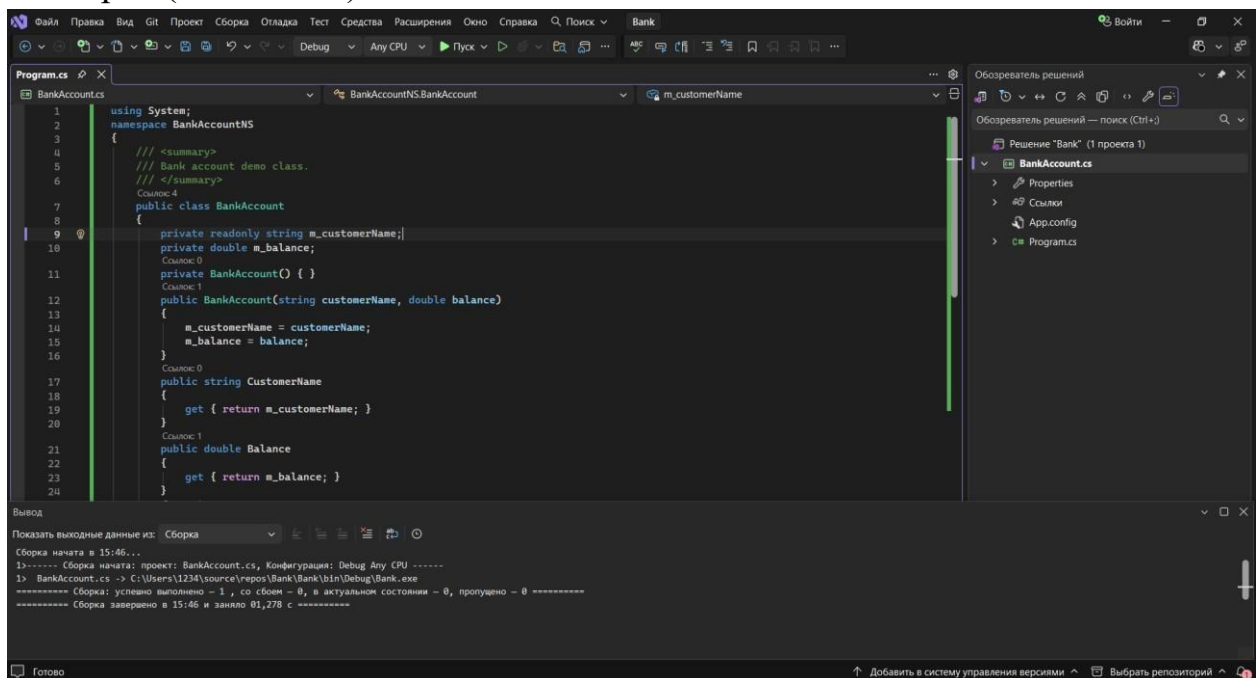


Рисунок 8

Результат моей работы:

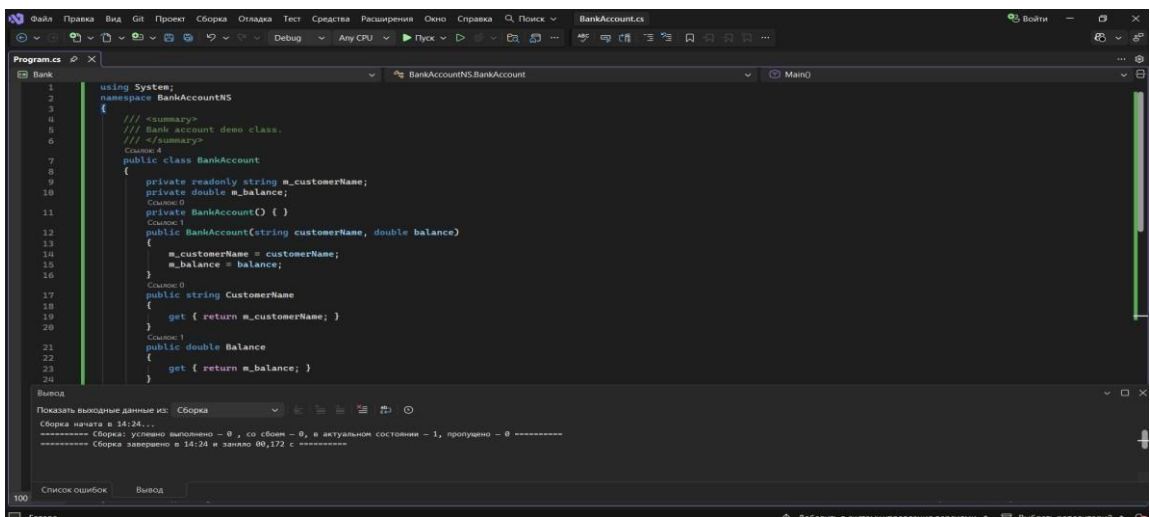


Рисунок 9

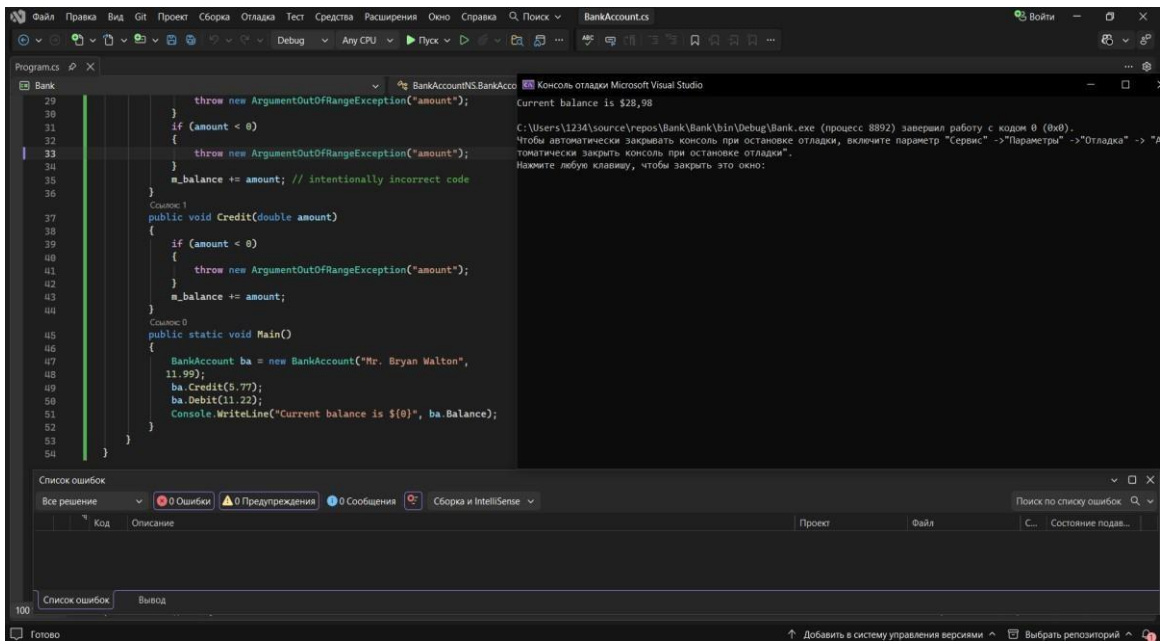


Рисунок 10

2. Создание проекта модульного теста.

1. В меню Файл я выбрала Добавить-Создать проект.

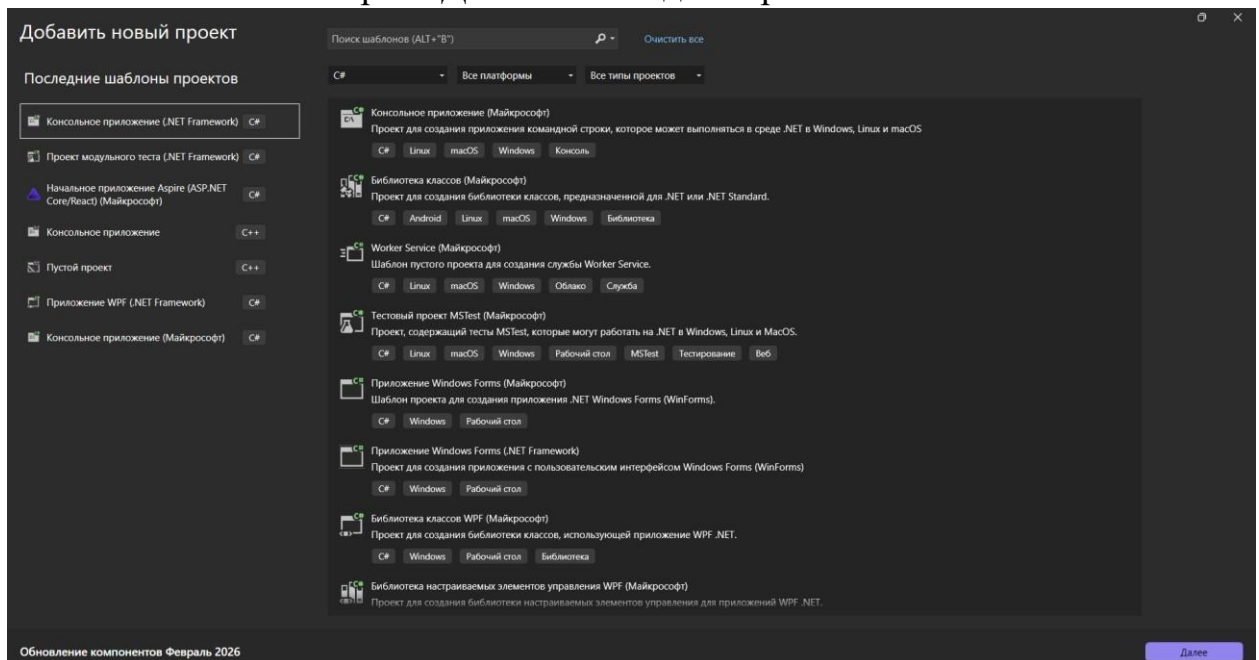


Рисунок 11

2. Я ввела test в поле поиска, выбрала C# в качестве языка, затем выбрала Проект модульного теста MSTest (.NET Core) для C# в качестве шаблона .NET Core.

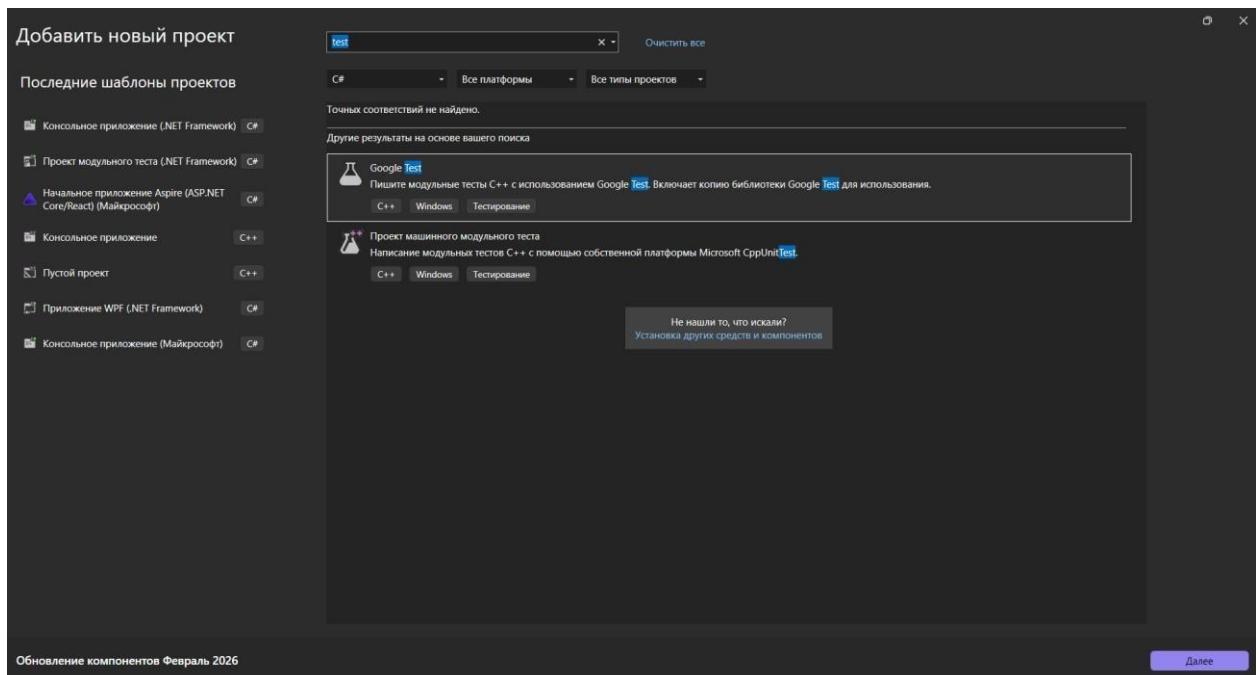


Рисунок 12

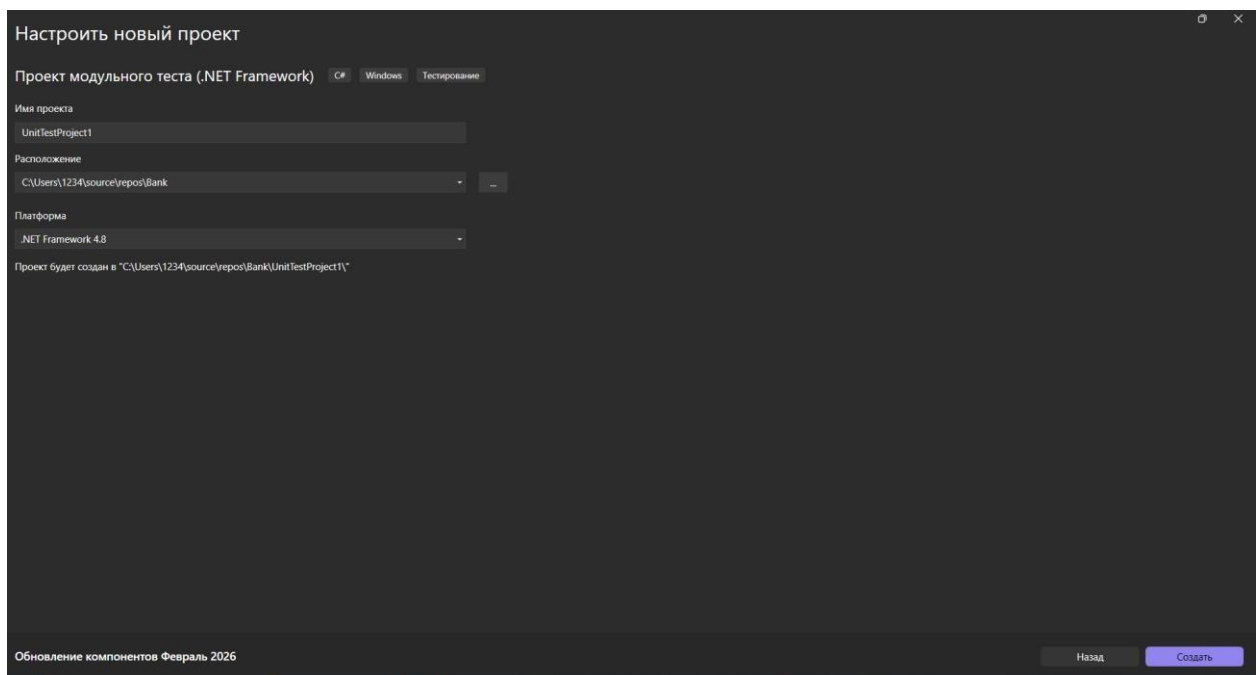


Рисунок 13

3. Я назвала проект BankTests.

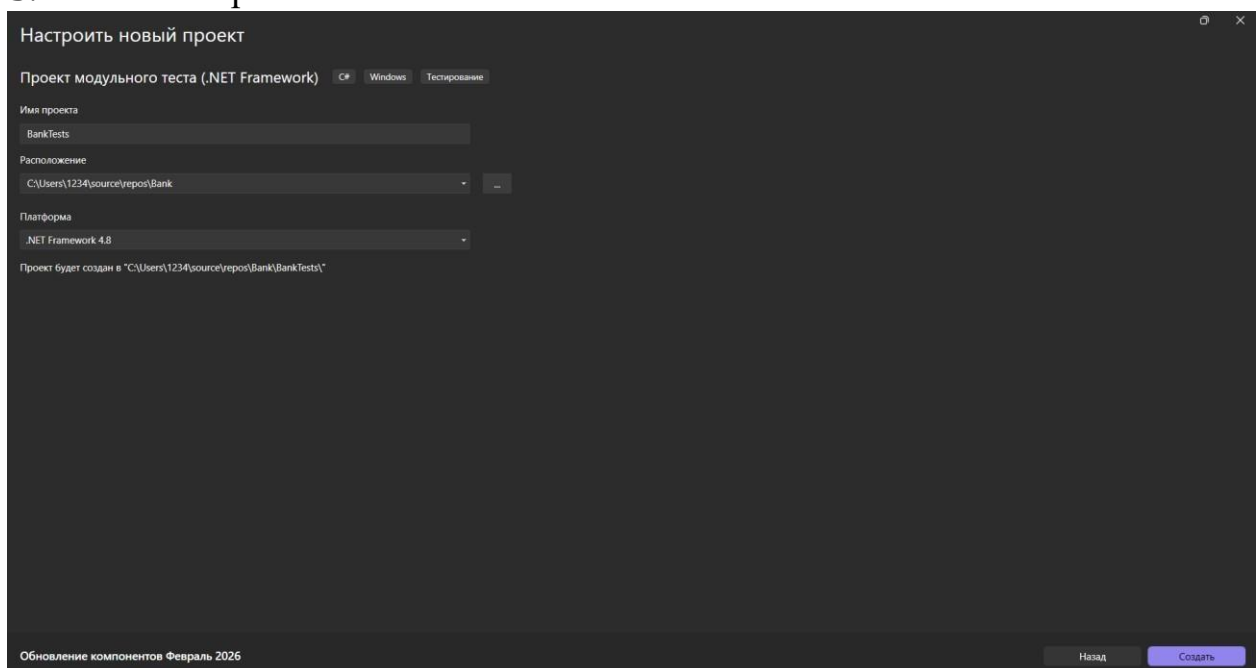


Рисунок 14

4. Я выбрала рекомендуемую версию целевой платформы.

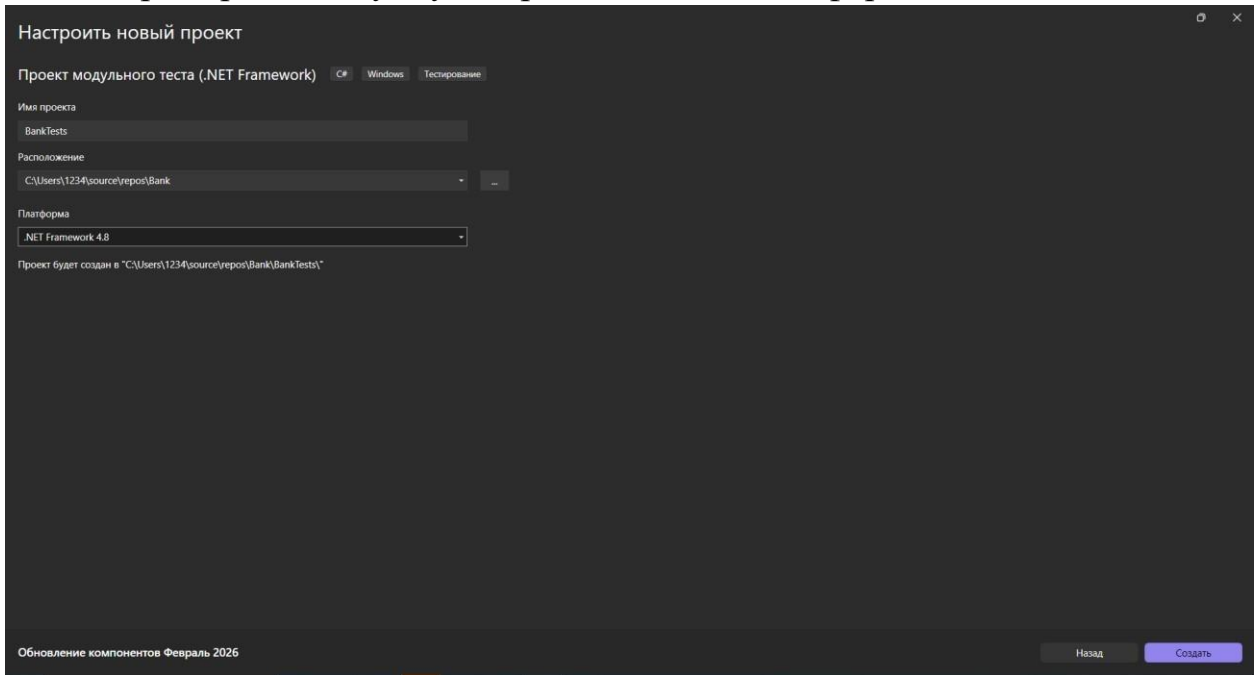


Рисунок 15

5. В проекте BankTests я добавила ссылку на проект Банк.

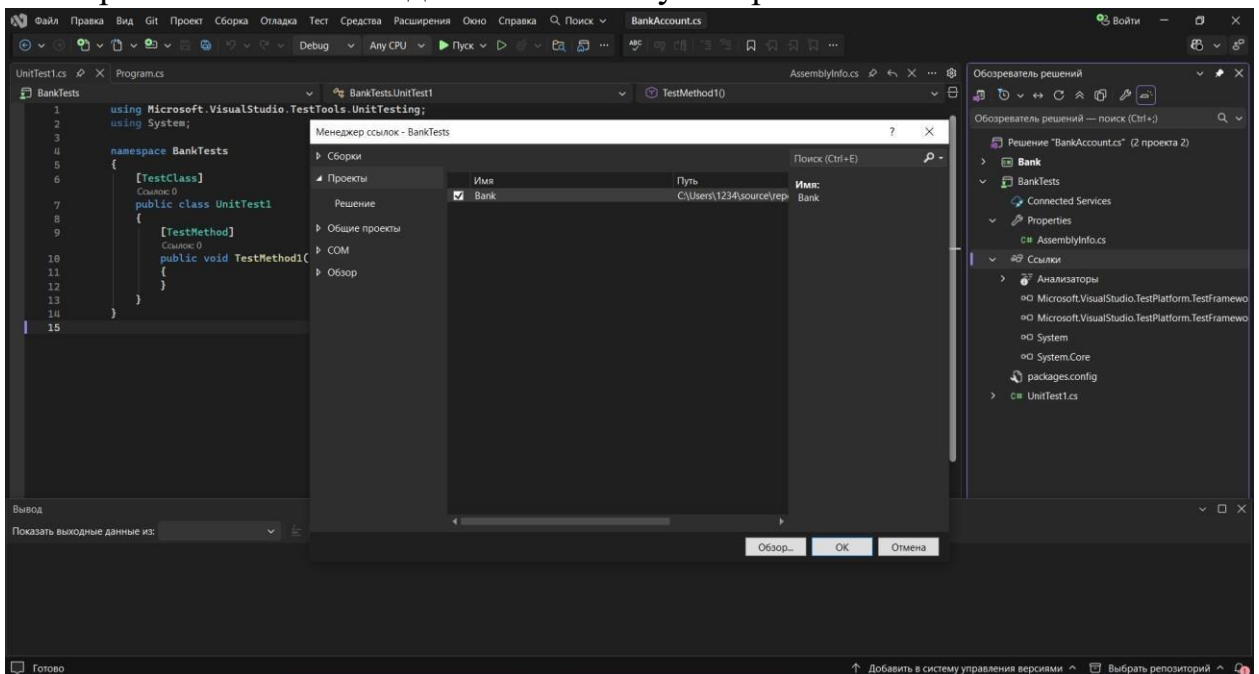


Рисунок 16

6. В диалоговом окне Диспетчер ссылок я развернула Проекты, выбрала Решение и выбрала элемент Банк и нажала кнопку ОК.

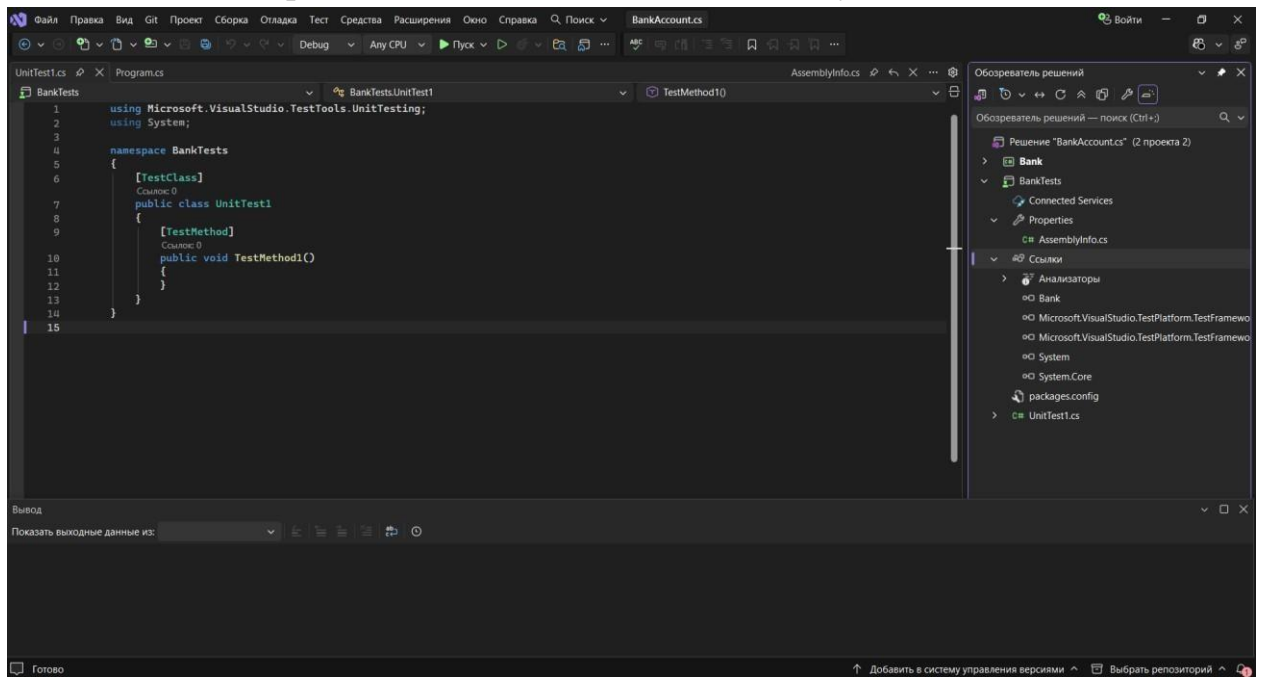


Рисунок 17

3. Создание тестового класса.

1. Я переименовала файл в BankAccountTests.cs.

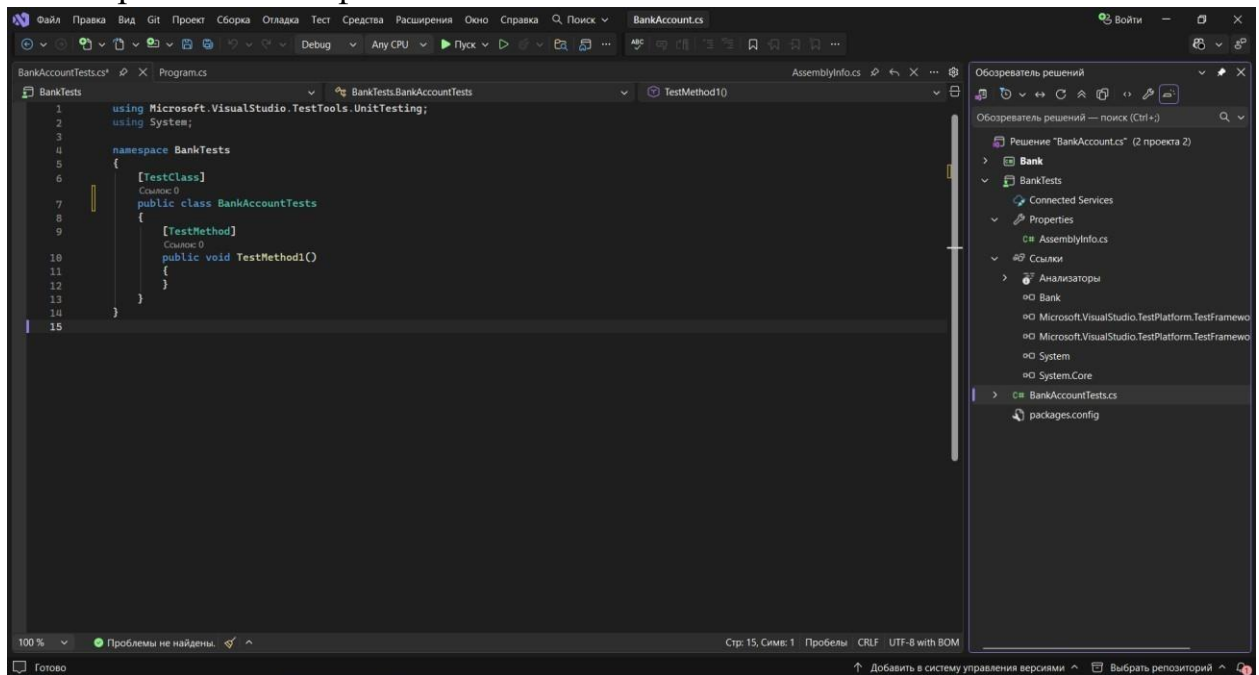


Рисунок 18

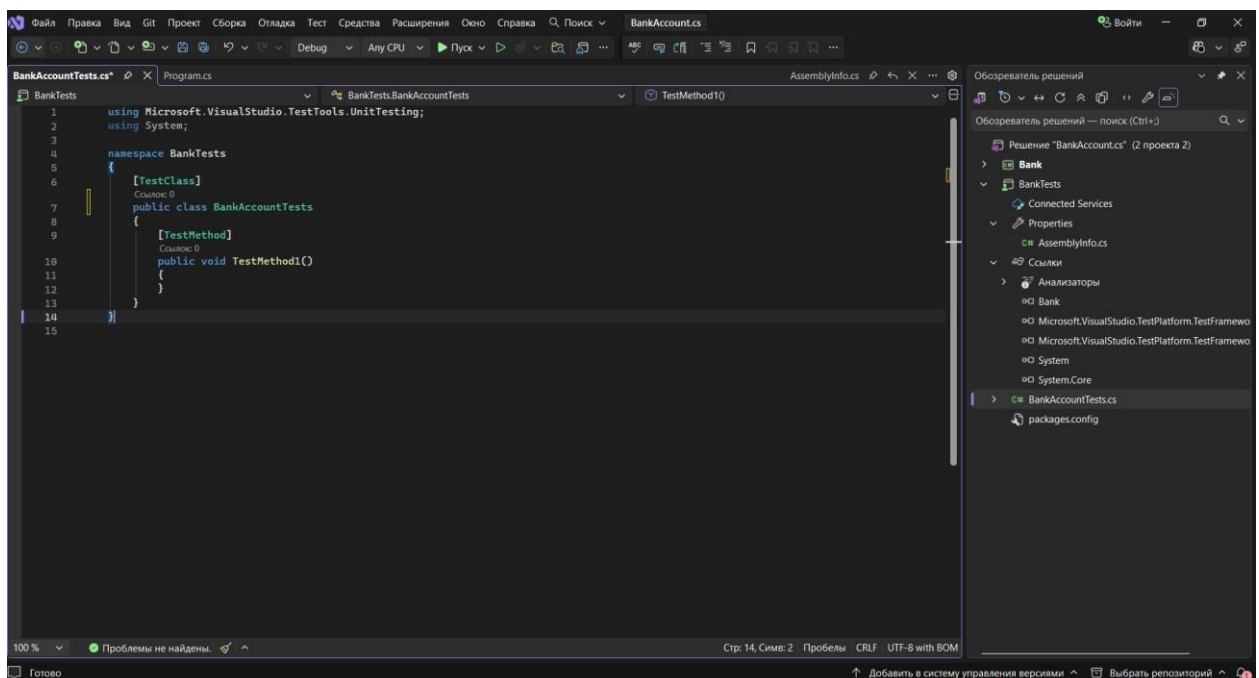


Рисунок 19

2. В начале файла тестового класса я прописала оператор using. Данное действие необходимо для того, чтобы иметь возможность вызывать методы основного проекта без необходимости каждый раз указывать их полное имя.

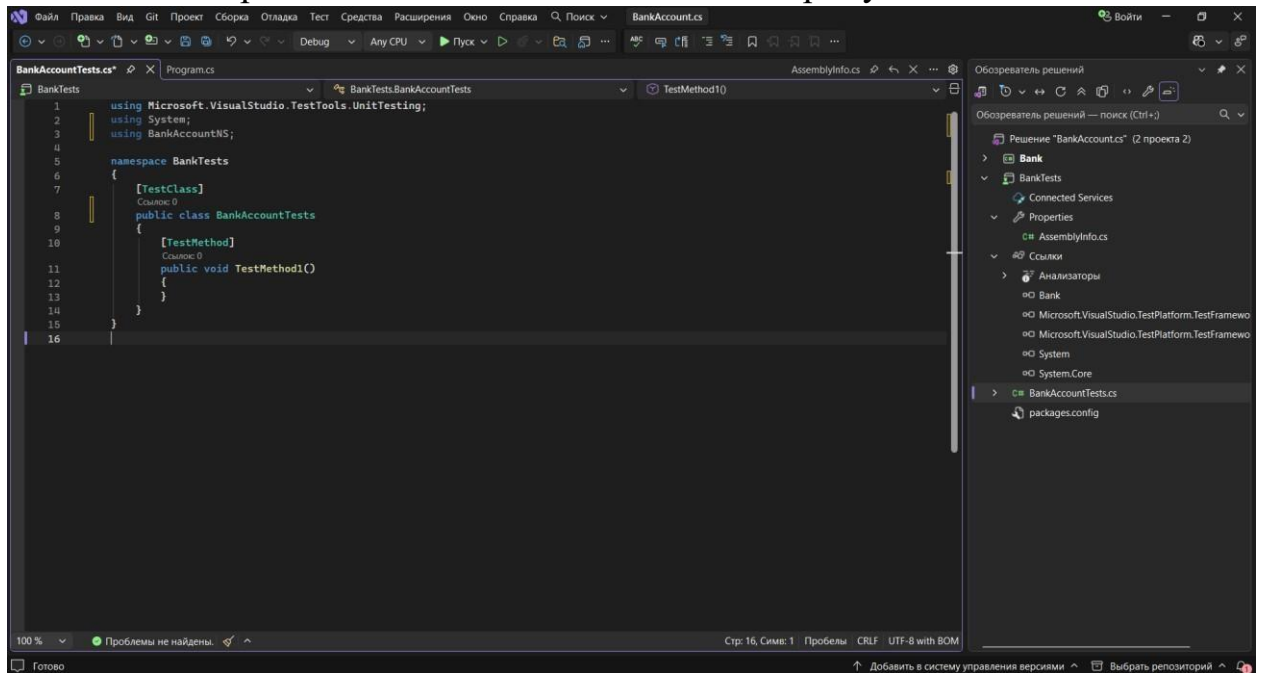


Рисунок 20

4. Создание метода теста.

1. Я ввела предложенный код.

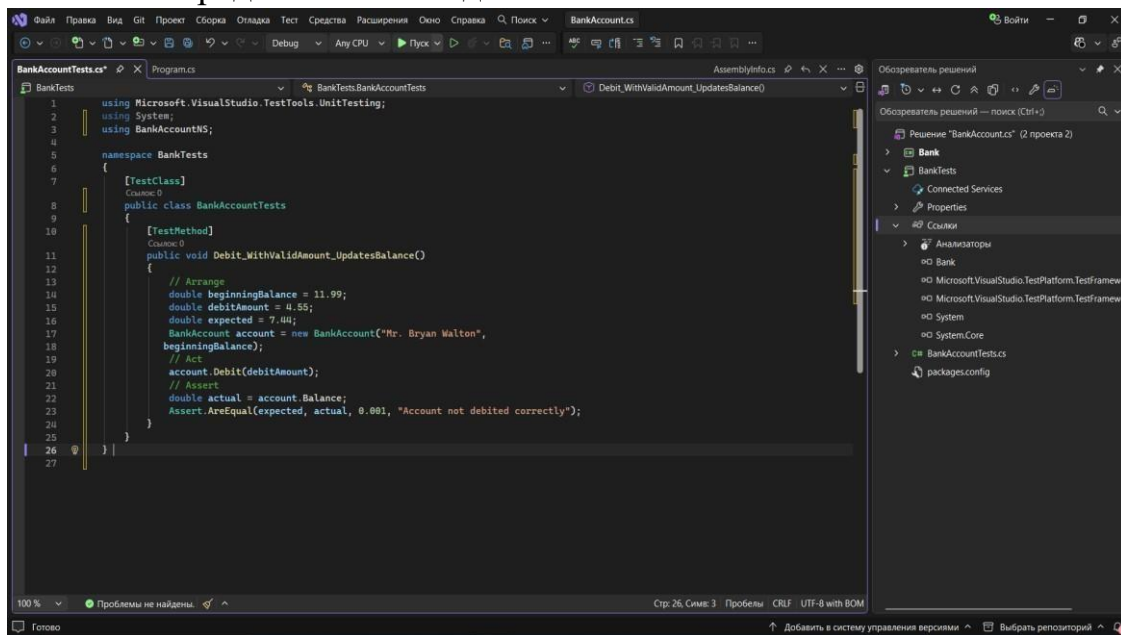


Рисунок 21

5. Сборка и запуск теста.

1. Я использовала клавиши CTRL +SHIFT + B для того, чтобы произошла сборка (вывод был).

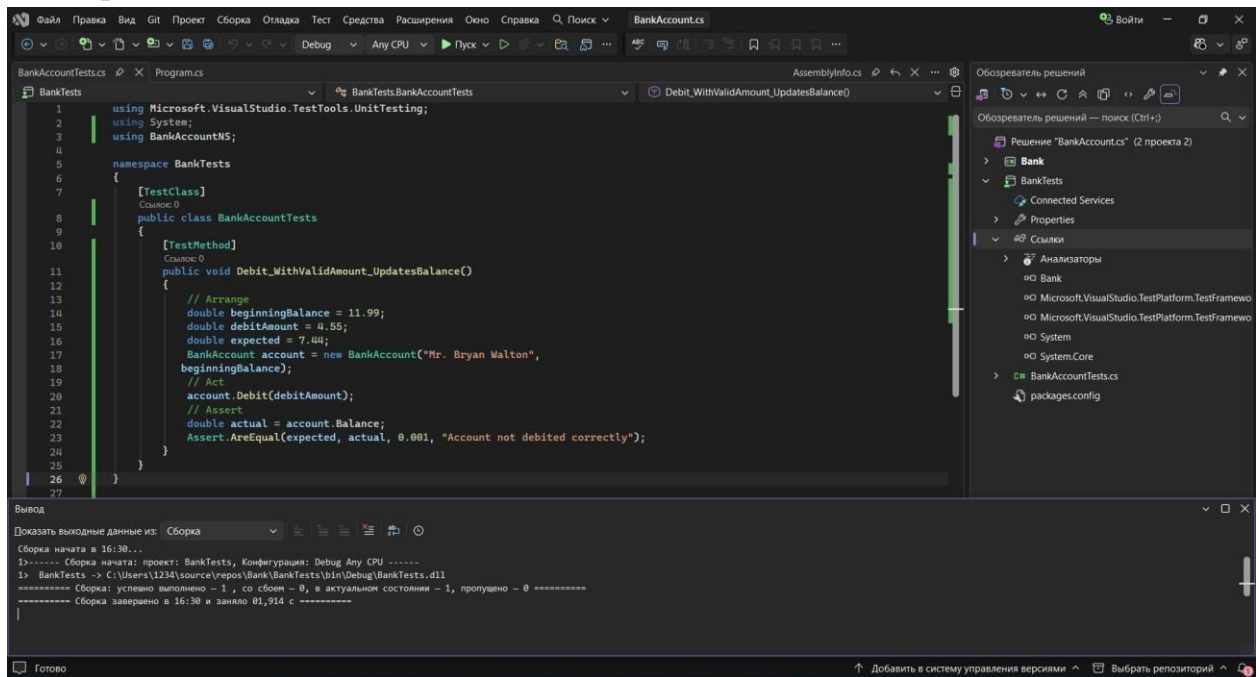


Рисунок 22

2. Я открыла Обзорщик тестов, выбрав Тест>Windows>Обзорщик тестов в верхней строке меню.

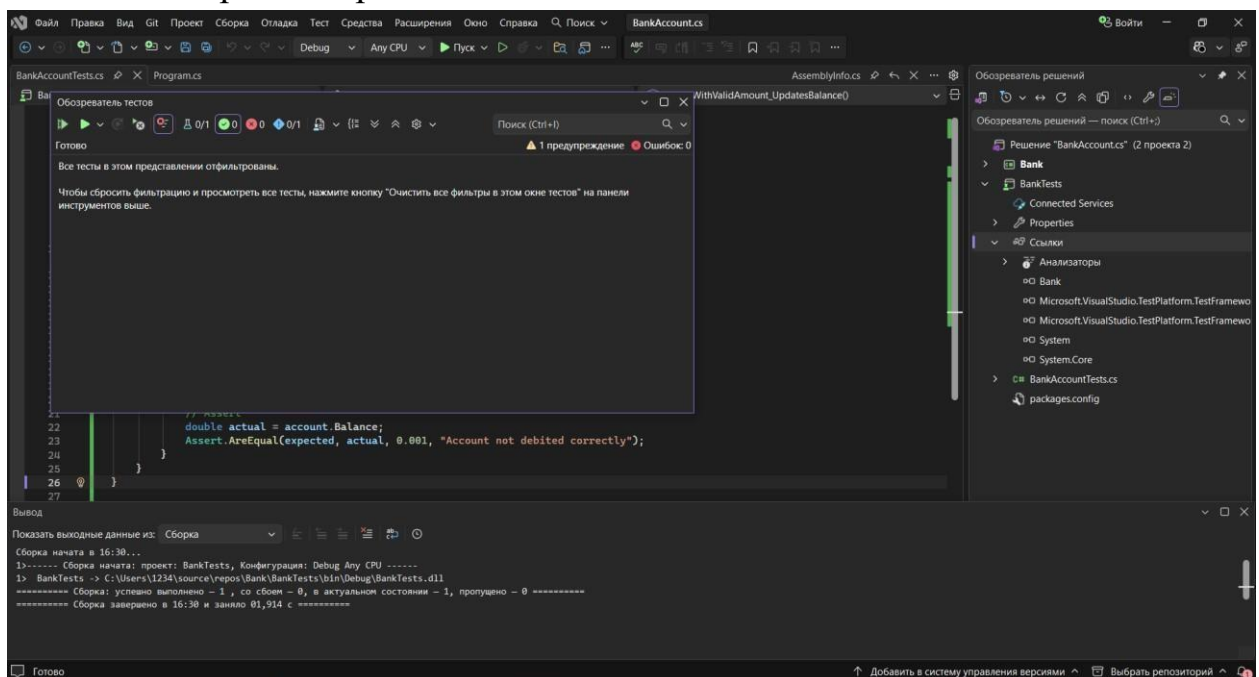


Рисунок 23

3. Я выбрала Запустить все, чтобы выполнить тест. В данном случае тест не пройден.

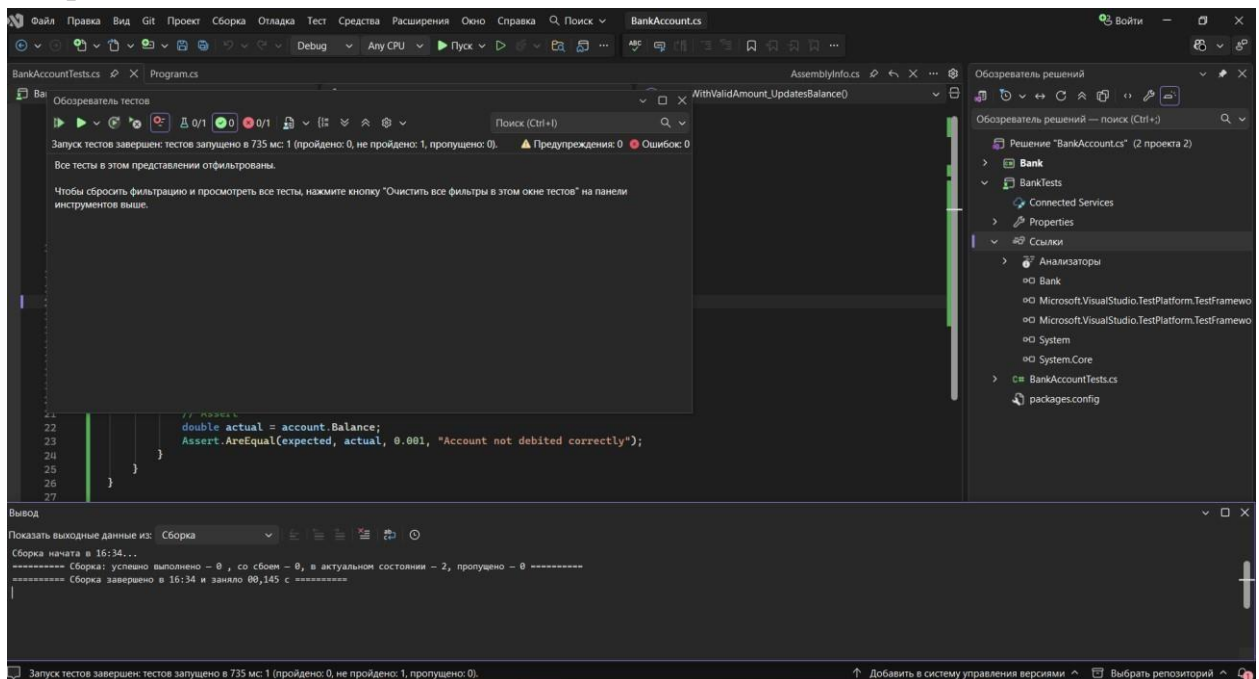


Рисунок 24

4. Я выбрала эту ошибку в обозревателе тестов для просмотра сведений в нижней части окна.

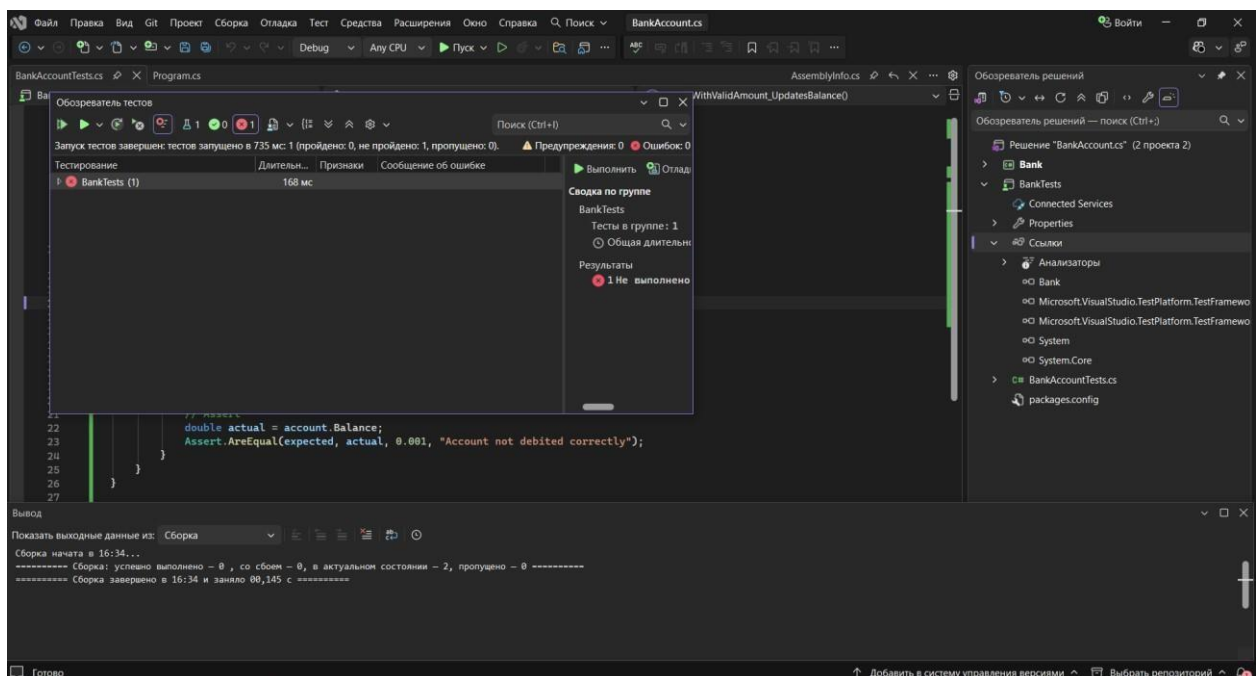


Рисунок 25

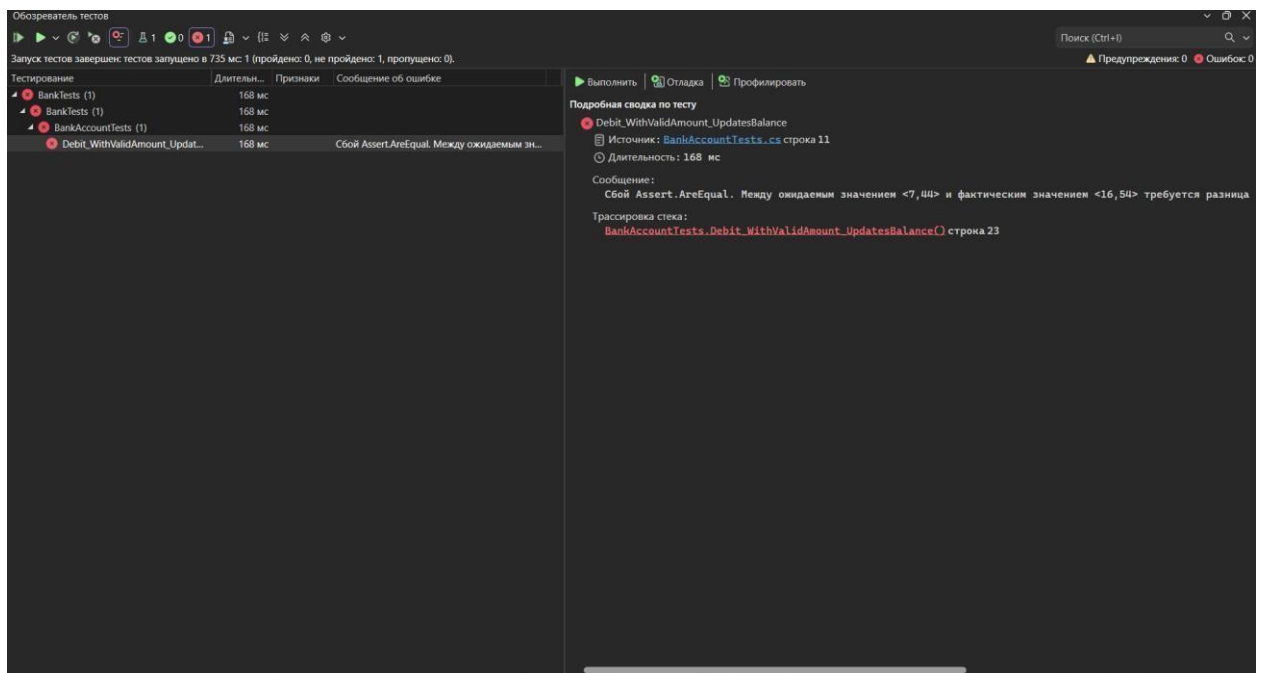


Рисунок 26

6. Исправление кода и повторный запуск тестов.

Я сделала исправление ошибки: В файле BankAccount.cs заменила строки с:

`m_balance += amount;`

на:

`m_balance -= amount;`

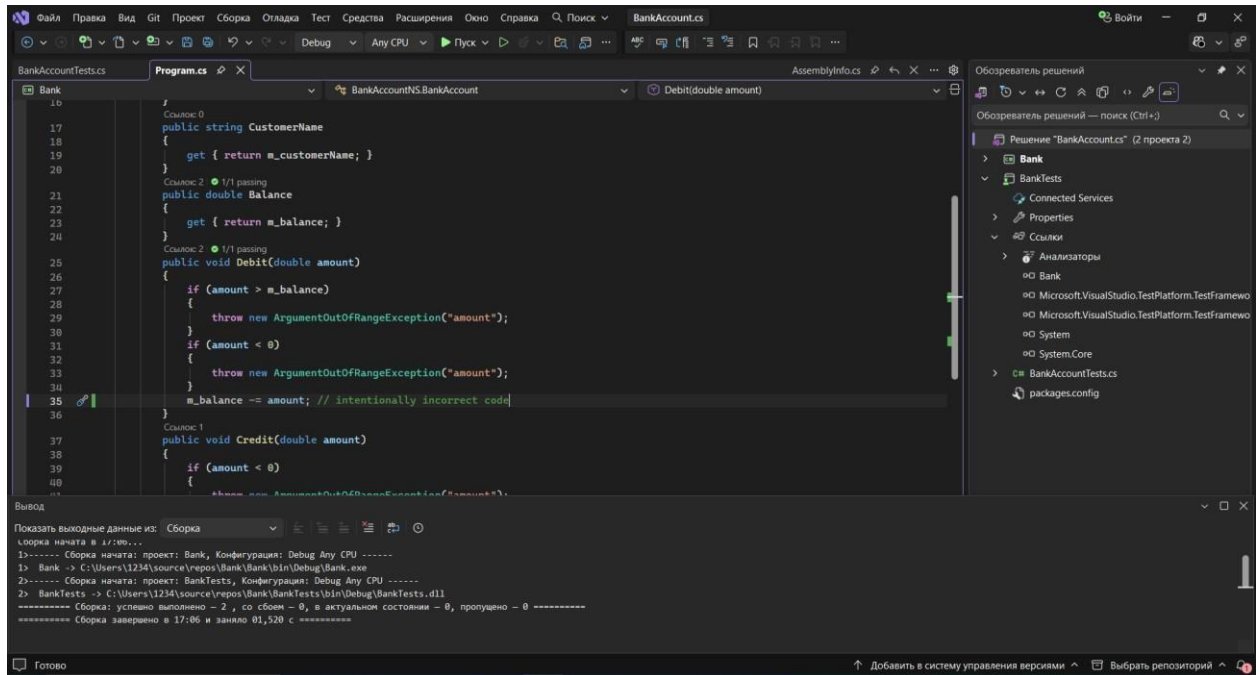


Рисунок 27

Я сделала повторный запуск теста.

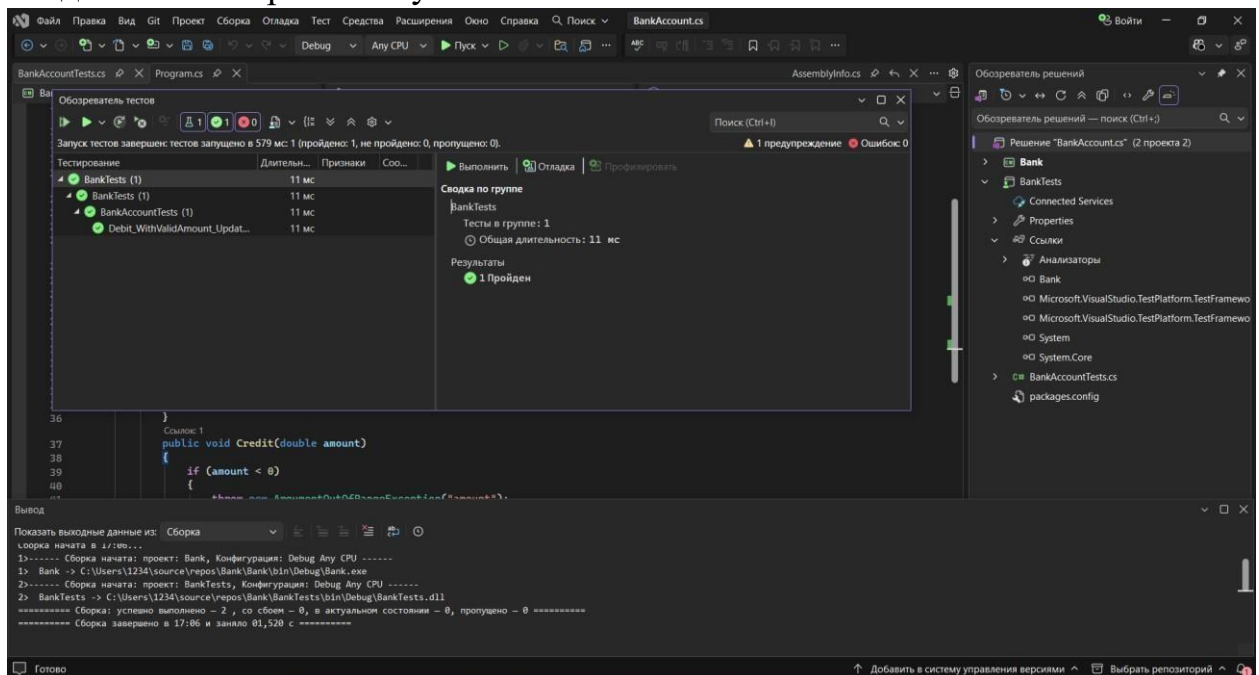


Рисунок 28

7. Создание и запуск новых методов теста.

1. Я ввела предложенный код.

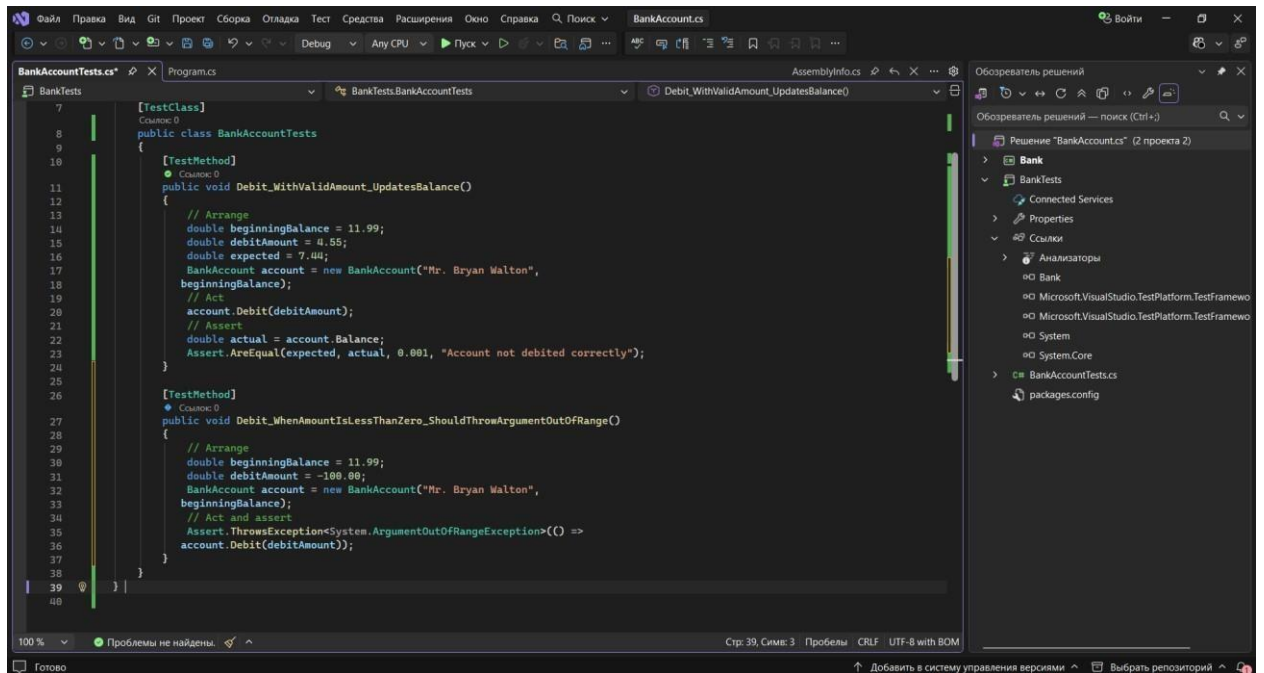


Рисунок 29

2. Выполнила проверку 2 тестов, убедилась, что они пройдены и нет ошибок.

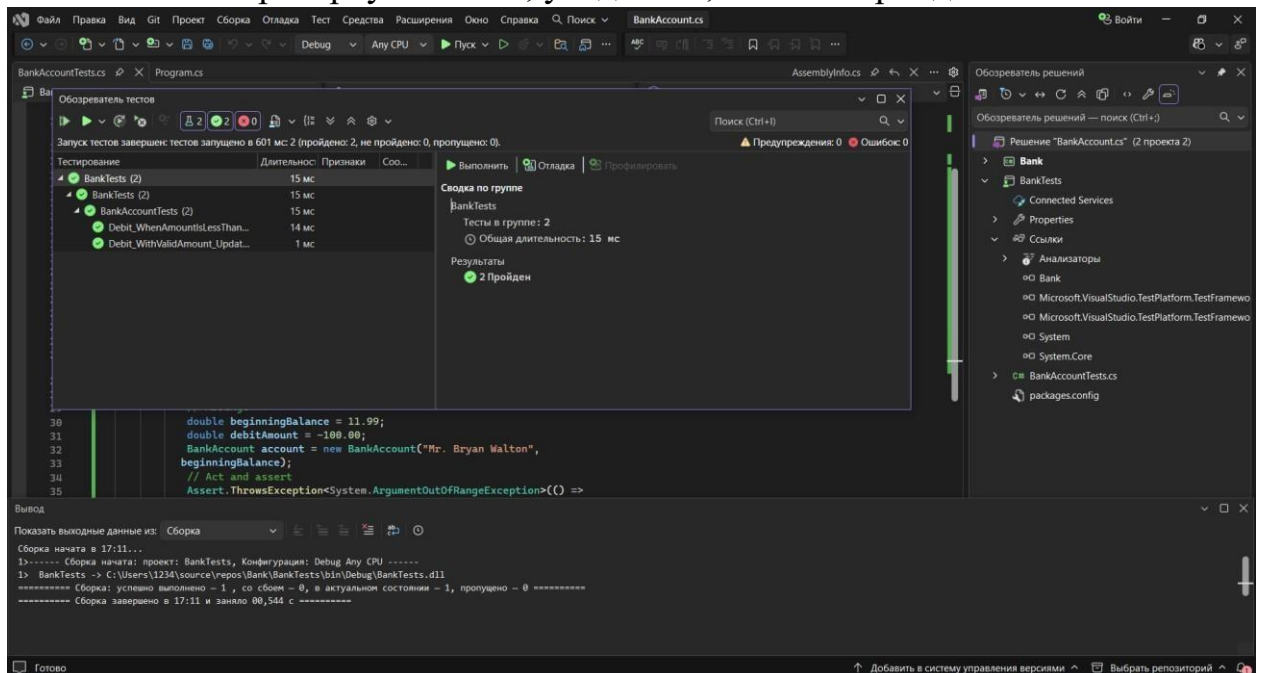


Рисунок 30

8. Рефакторинг тестируемого кода.

1. Я приступила к рефакторингу тестового кода. Это позволит значительно улучшить диагностику: теперь в случае неудачи я смогу видеть конкретную причину сбоя теста, а не просто сам факт ошибки.

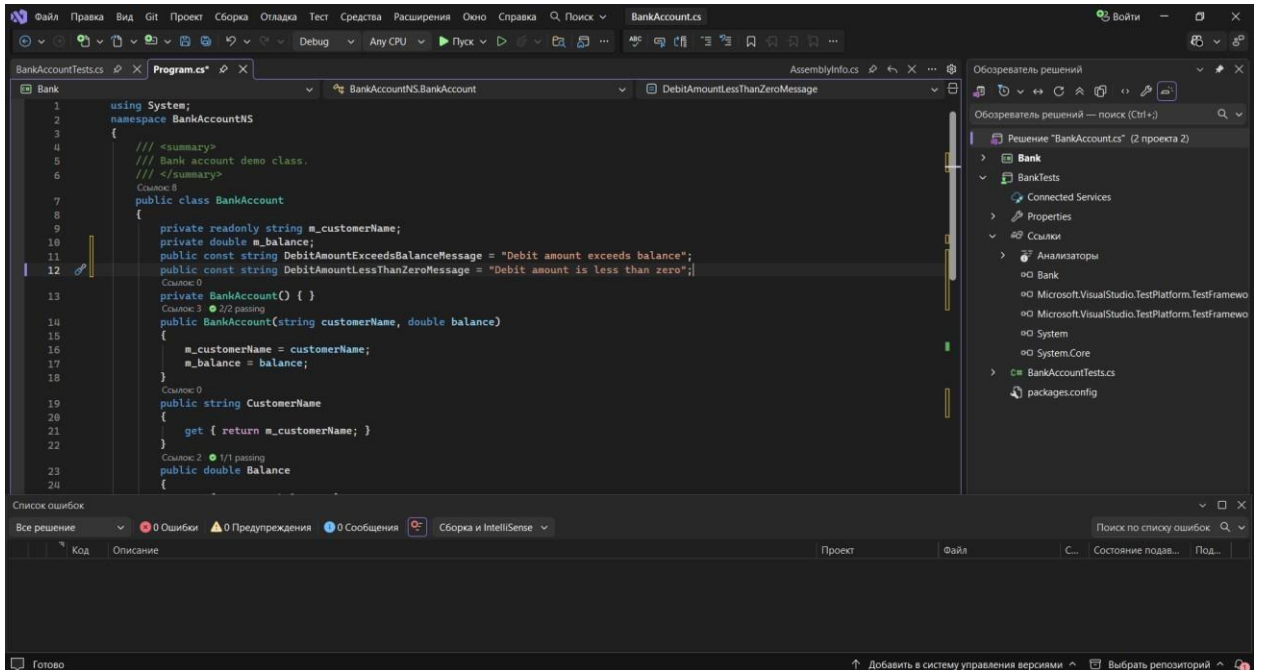


Рисунок 31

2. Я сделала замену двух условных операторов в методе Debit.

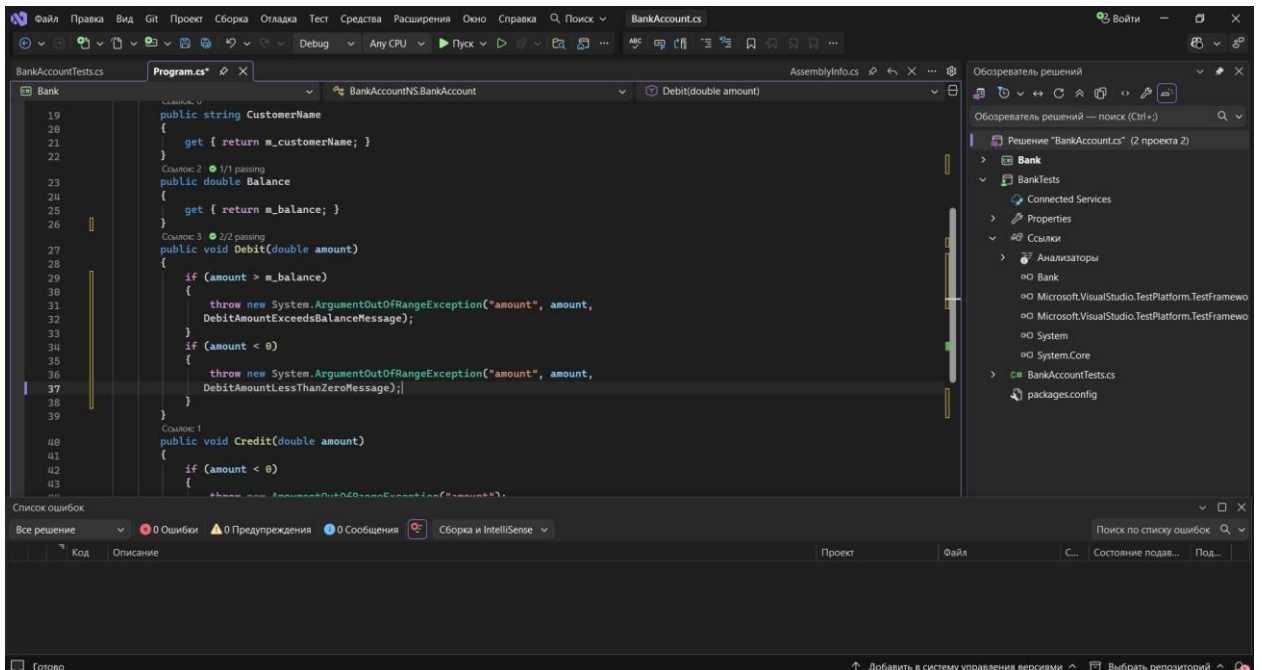


Рисунок 32

9. Рефакторинг тестовых методов.

1. Я ввела предложенный код:

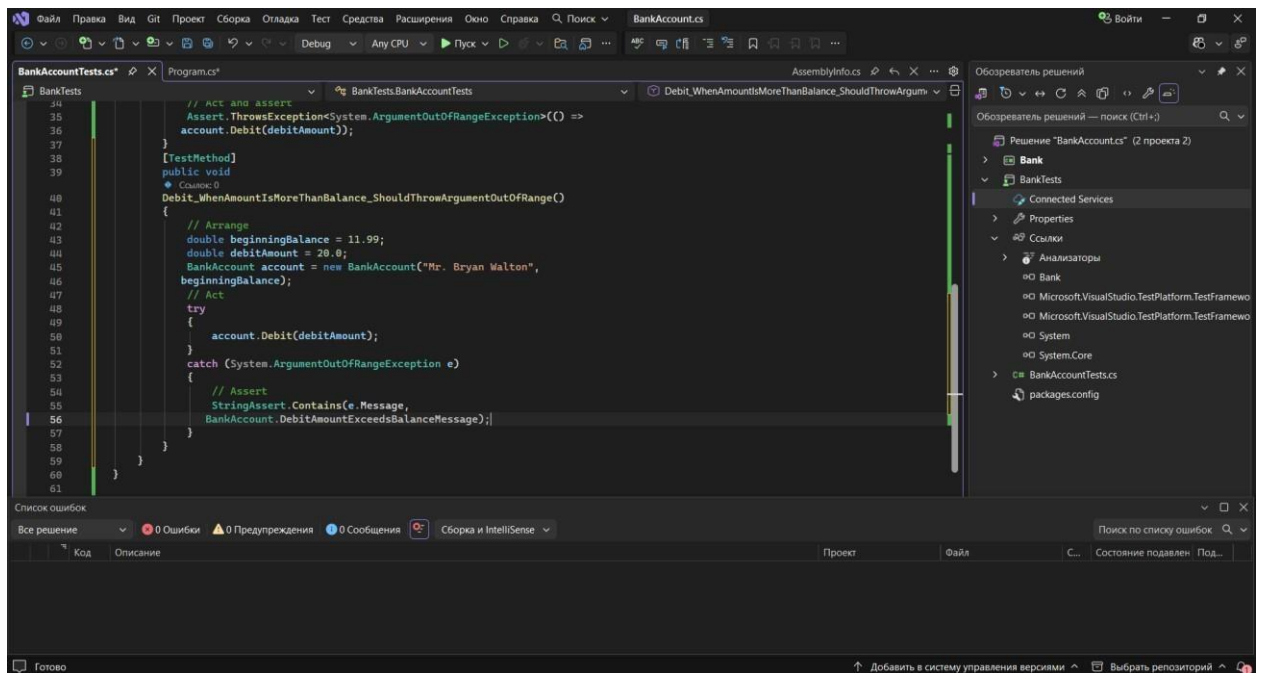


Рисунок 33

10. Повторное тестирование, переписывание и анализ.

1. Я отредактировала код.

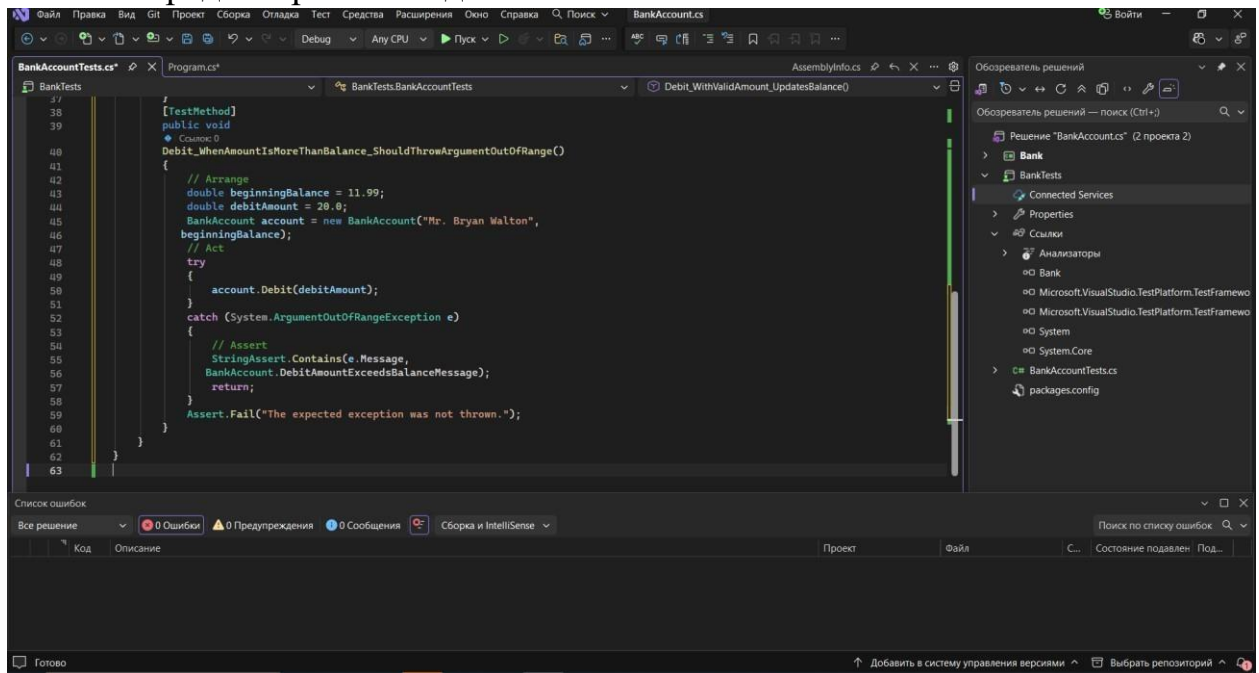


Рисунок 34

2. Я провела проверку тестов. Я обнаружила ошибки в одном из тестов.

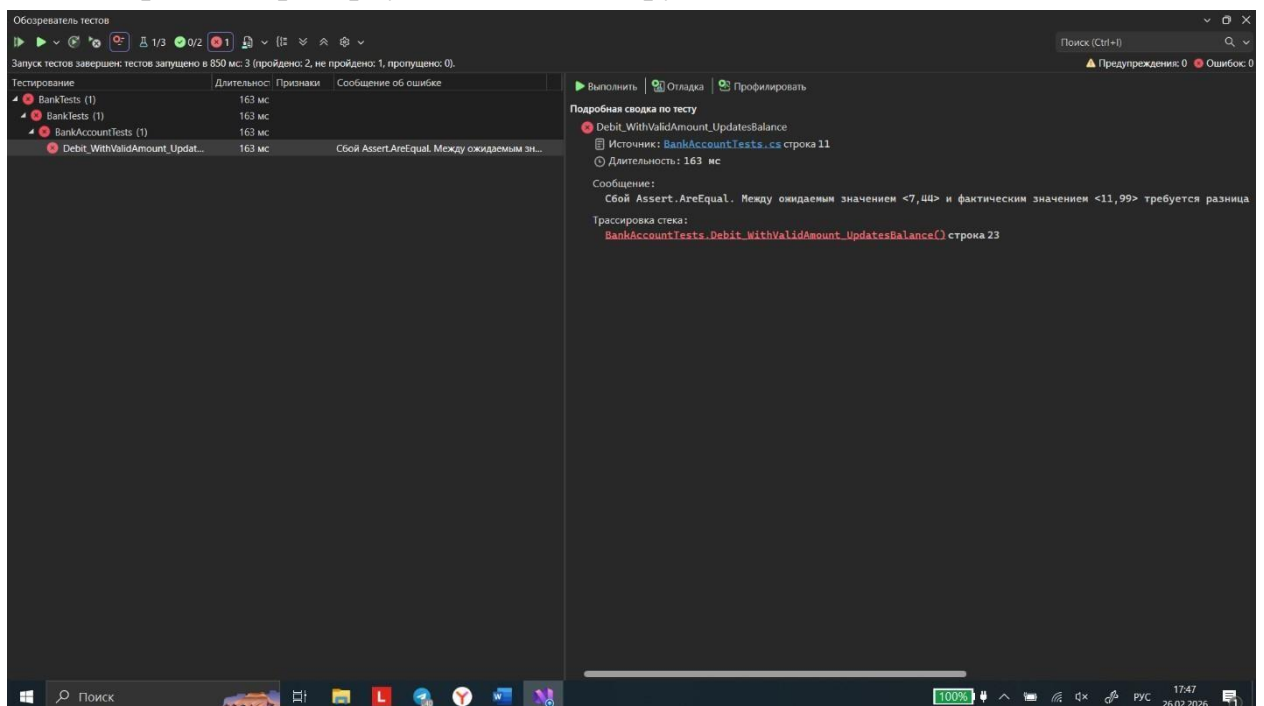


Рисунок 35

3. Я приняла решение анализировать ошибку и искать причины её возникновения, и пыталась решить возникшие ошибки.

Я добавила строку в код для устранения ошибки: `m_balance -= amount;`

4. Я совершила запуск всех тестов (повторная проверка).

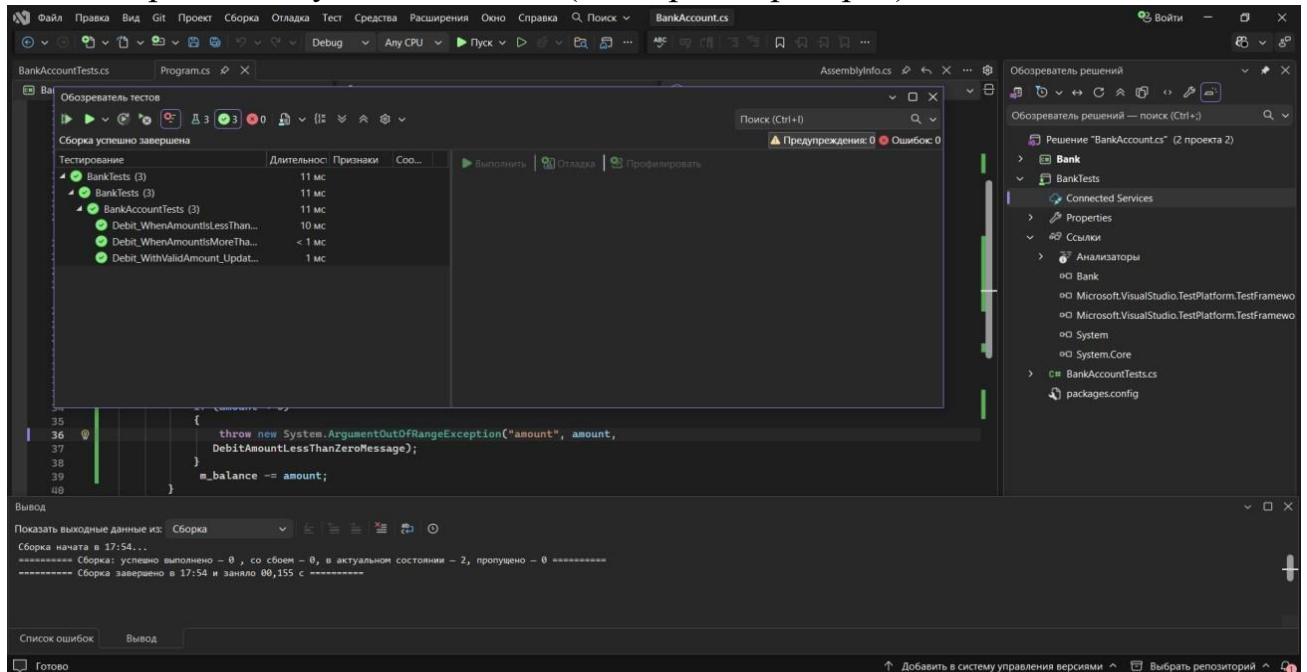


Рисунок 36

5. Я провела анализирование вывода, ошибка устранена. Все тесты выполнены правильно.

6. Проверка работы программы.

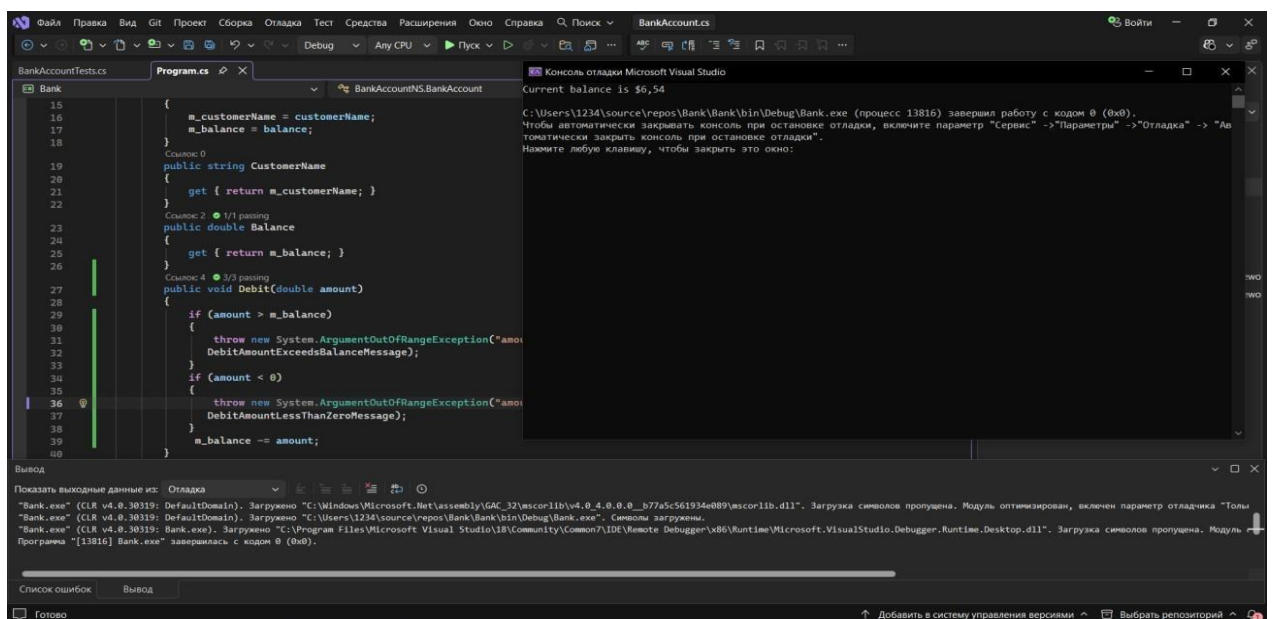


Рисунок 37

Заключение.

Выполняя практическую работу №1 по теме «Создание и запуск модульных тестов для управляемого кода», я освоила работу со средой тестирования. Научилась создавать и запускать модульные тесты, и выполнять рефакторинг. Таким образом, я смогла усовершенствовать тестовый код, что привело к созданию более надёжного информативного метода теста. Но самое главное, что в результате был улучшен тестируемый код. Я научилась составлять официальную документацию. Осознала, что всегда нужно перепроверять свою работу множество раз и что заниматься тестированием очень классно.

